



VAERDIA

Plateforme de Gestion de Projets Agiles

Rapport de Projet de Fin d'Études

Spécifications Fonctionnelles et Architecture Système

Version 1.0

18 avril 2026

VAERDIA
Division Technologies et Innovation

Abstract

This report presents the end-of-studies project carried out within **VAERDIA**, a Tunisian company specializing in digital innovation and custom software development, located in Sousse, Tunisia.

The objective of this project is the design and development of an **internal web platform for agile project management**, tailored to VAERDIA's operational needs. The proposed solution centralizes all business processes within a unified, scalable, and intelligent system hosted on VAERDIA's own infrastructure.

The platform is built around the **Scrum** methodology and integrates eight functional modules : project management, task management, document management, internal communication, client portal and ticketing, leave and HR management, reporting and dashboards, as well as an intelligent assistant named **Aura AI**.

From a technical standpoint, the adopted architecture is **microservices**-based, with a **React / TypeScript** frontend, a **Nest.js** backend API Gateway, an AI module developed in **FastAPI (Python)** leveraging **LangChain** and the **OpenAI API**, data persistence via **PostgreSQL**, file storage via **MinIO (S3-compatible)**, and a fully containerized deployment using **Docker Compose** with a **GitLab CI/CD** pipeline.

Keywords : Agile project management, Scrum, Artificial intelligence, Microservices, React, Nest.js, FastAPI, LangChain, PostgreSQL, Docker.

Résumé

Ce rapport présente le projet de fin d'études réalisé au sein de l'entreprise **VAERDIA**, société tunisienne spécialisée dans l'innovation numérique et le développement de solutions logicielles sur mesure, dont le siège est établi à Sousse, Tunisie.

L'objectif de ce projet est la conception et le développement d'une **plateforme web interne de gestion de projets agiles**, répondant aux besoins opérationnels de VAERDIA. La solution proposée centralise l'ensemble des processus métiers au sein d'un système unifié, évolutif et intelligent, hébergé sur l'infrastructure propre de l'entreprise.

La plateforme repose sur la méthodologie **Scrum** et intègre huit modules fonctionnels, ainsi qu'un assistant intelligent nommé **Aura IA**, capable de détecter des anomalies, de générer des rapports automatiques et de répondre à des requêtes en langage naturel.

Du point de vue technique, l'architecture adoptée est de type **microservices**, avec un frontend **React / TypeScript**, un backend **Nest.js** en tant qu'API Gateway, un module IA développé en **FastAPI (Python)**, une persistance assurée par **PostgreSQL** et un stockage de fichiers via **MinIO**, le tout déployé via **Docker Compose** et un pipeline **GitLab CI/CD**.

Mots-clés : Gestion de projets agiles, Scrum, Intelligence artificielle, Microservices, React, Nest.js, FastAPI, LangChain, PostgreSQL, Docker.

Remerciements

Au terme de ce projet de fin d'études, nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce travail.

Nous adressons nos remerciements les plus sincères à :

M. Malek Ben Salem, notre encadrant académique, pour sa disponibilité, ses précieux conseils et son accompagnement rigoureux tout au long de ce projet.

M. Achref Makhloufi, notre encadrant professionnel au sein de VAERDIA, pour son accueil chaleureux, sa confiance et son engagement à nos côtés durant toute la durée du stage.

Nous remercions également l'ensemble de l'équipe de **VAERDIA** pour leur accueil et les échanges enrichissants qui ont nourri notre réflexion au quotidien.

Enfin, nous exprimons notre reconnaissance à nos enseignants et au corps pédagogique pour la formation de qualité dispensée, ainsi qu'à notre famille et nos proches pour leur soutien moral indéfectible.

Merci à tous.

Table des matières

Abstract	1
Résumé	2
Remerciements	3
Liste des figures	6
Liste des tableaux	7
1 Contexte général du projet	8
Introduction	8
1.1 Présentation de l'organisme d'accueil	8
1.1.1 VAERDIA	8
1.1.2 Organisation générale	9
1.2 Présentation générale du projet	9
1.3 Problématique	9
1.4 Analyse de l'existant	10
1.4.1 Description de l'existant	10
1.4.2 Critique de l'existant	10
1.4.3 Solution proposée	10
1.5 Solutions existantes	11
1.6 Méthodologie et formalisme adoptés	12
1.6.1 Le choix de la méthodologie Scrum	12
Conclusion	13
2 Pilotage du projet avec Agile Scrum	14
Introduction	14
2.1 Identification des acteurs	14
2.2 Product Backlog	15
2.2.1 Module : Authentification et accès	15
2.2.2 Module : Gestion des projets	16
2.2.3 Module : Gestion des tâches (Scrum)	16
2.2.4 Module : Portail client, Documents & Communication	17
2.2.5 Module : RH, Reporting & Aura IA	18
2.3 Planification des releases et sprints	19
2.3.1 Principes de planification	19
2.3.2 Planification des releases	20
2.3.3 Planning détaillé des sprints	20
2.4 Diagramme de Gantt	21
2.5 Besoins non fonctionnels	21

2.6	Architecture logicielle et physique	22
2.6.1	Architecture logicielle	22
2.6.2	Architecture physique	23
2.7	Environnement technique de travail	24
2.7.1	Environnement de développement Frontend	24
2.7.2	Environnement de développement Backend	24
2.7.3	Outils de collaboration et de gestion de projet	25
	Conclusion	26
3	Conception et réalisation du Release 1	27
	Introduction	27
3.1	Sprint 1 : Authentification	27
3.1.1	Backlog du sprint 1	27
3.1.2	Spécification des besoins du sprint 1	28
3.1.3	Conception du sprint 1	30
3.1.4	Implémentation du sprint 1	33
3.2	Sprint 2 : Gestion des projets	36
3.2.1	Backlog du sprint 2	37
3.2.2	Spécification des besoins du sprint 2	37
3.2.3	Conception du sprint 2	38
3.2.4	Implémentation du sprint 2	40
3.3	Sprint 3 : Backlog & Scrum	42
3.3.1	Backlog du sprint 3	43
3.3.2	Spécification des besoins du sprint 3	43
3.3.3	Conception du sprint 3	47
3.3.4	Implémentation du sprint 3	48
	Conclusion	53
	Références bibliographiques	54

Table des figures

1.1	Logo de VAERDIA	9
1.2	Solutions de gestion de projet analysées sur le marché	11
1.3	Cycle de développement Scrum [1]	12
2.1	Diagramme de Gantt — VAERDIA (16 semaines / 4 mois)	21
2.2	Architecture logicielle de VAERDIA — vue en couches	23
2.3	Architecture physique de VAERDIA	23
3.1	Diagramme de cas d'utilisation — Sprint 1	28
3.2	Diagramme de classes du sprint 1	31
3.3	Diagramme de séquence « Créer un utilisateur »	32
3.4	Diagramme de séquence « Se connecter »	33
3.5	Interface « Accueil »	34
3.6	Interface « Se connecter »	35
3.7	Interface « Gestion des utilisateurs »	36
3.8	Diagramme de cas d'utilisation « Gérer les projets »	37
3.9	Diagramme de classes du sprint 2	39
3.10	Diagramme de séquence « Créer un projet »	40
3.11	Interface « Créer un projet »	41
3.12	Interface « Dashboard projets »	42
3.13	Diagramme de cas d'utilisation « Gérer le backlog »	44
3.14	Diagramme de cas d'utilisation « Gérer un sprint »	45
3.15	Diagramme de classes du sprint 3	48
3.16	Interface « Product Backlog »	49
3.17	Interface « Créer une User Story »	50
3.18	Interface « Board Scrum »	51
3.19	Interface « Burndown Chart »	52
3.20	Interface « Clôture de sprint »	53

Liste des tableaux

1.1	Fiche d'identité de VAERDIA	8
1.2	Analyse comparative des solutions existantes sur le marché	11
1.3	Artefacts Scrum	13
1.4	Composition de l'équipe Scrum du projet VAERDIA	13
2.1	Identification et synthèse des acteurs du système VAERDIA	15
2.2	Backlog — Module Authentification et accès	16
2.3	Backlog — Module Gestion des projets	16
2.4	Backlog — Module Gestion des tâches / Scrum	17
2.5	Backlog — Module Portail Client, Documents & Communication	18
2.6	Backlog — Modules RH, Reporting, Aura IA & Déploiement	19
2.7	Planification des deux releases du projet VAERDIA (16 semaines / 4 mois)	20
2.8	Planning détaillé des 6 sprints — VAERDIA (16 semaines / 4 mois)	20
2.9	Environnement de développement Frontend	24
2.10	Environnement de développement Backend	25
3.1	Backlog du sprint 1 — Authentification & Infrastructure	27
3.2	Description textuelle — « Créer un utilisateur »	29
3.3	Description textuelle — « Se connecter »	30
3.4	Backlog du sprint 2 — Gestion des projets	37
3.5	Description textuelle — « Créer un projet »	38
3.6	Backlog du sprint 3 — Backlog & Scrum	43
3.7	Description textuelle — « Gérer le backlog »	46
3.8	Description textuelle — « Planifier un sprint »	47

Chapitre 1

Contexte général du projet

Introduction

Ce chapitre établit les bases de notre projet en explorant trois dimensions clés. Dans un premier temps, nous présentons VAERDIA, l'entreprise qui nous accueille, en détaillant son positionnement sur le marché ainsi que son organisation interne. Ensuite, nous abordons la question centrale : comment élaborer une plateforme unifiée qui réponde aux exigences de gestion de projets agiles tout en intégrant l'intelligence artificielle ? Pour finir, une étude de l'existant, enrichie par une analyse comparative du marché, nous permettra de définir les besoins fonctionnels et de justifier nos choix méthodologiques et techniques.

1.1 Présentation de l'organisme d'accueil

Cette section présente l'entreprise VAERDIA, son secteur d'activité et son organisation interne, afin de mieux situer le contexte dans lequel ce projet a été mené.

1.1.1 VAERDIA

VAERDIA est une entreprise tunisienne dédiée à l'innovation numérique et au développement de solutions logicielles sur mesure. Basée à Sousse, elle opère principalement dans trois secteurs stratégiques : la gestion de projets, l'automatisation des processus d'affaires et l'application de l'intelligence artificielle. Le tableau suivant synthétise les informations clés de l'entreprise.

Champ	Information
Raison sociale	Vaerdia
Secteur d'activité	Solutions logicielles et transformation digitale
Siège social	Avenue Yasser Arafat, Sahloul, 4000 Sousse, Tunisie
Téléphone	+216 51 124 000 / +216 73 368 132
Email	contact@vaerdia.com
Site Web	www.vaerdia.com

TABLE 1.1 – Fiche d'identité de VAERDIA



FIGURE 1.1 – Logo de VAERDIA

Domaines d'intervention :

- Développement de plateformes web et mobiles sur mesure
- Solutions de gestion de projets et d'équipes agiles
- Intégration d'intelligence artificielle dans les processus métiers
- Consulting en architecture logicielle et DevOps
- Formation et accompagnement à la transformation digitale

1.1.2 Organisation générale

VAERDIA est organisée selon une structure matricielle combinant savoir-faire technique et flexibilité opérationnelle. L'entreprise s'articule autour de quatre domaines d'expertise : le développement web et mobile, l'intelligence artificielle et le machine learning, le design UI/UX, et l'infrastructure DevOps. Elle encourage une culture agile forte, avec des cycles itératifs courts et l'adoption généralisée des méthodologies Scrum et DevOps au sein de ses équipes.

1.2 Présentation générale du projet

Cette section décrit les grandes lignes du projet VAERDIA, ses objectifs principaux et la vision fonctionnelle qui a guidé sa conception.

VAERDIA est une plateforme web interne développée afin d'améliorer la gestion des projets et des équipes au sein de l'entreprise. Elle vise à centraliser la planification des projets, le suivi des tâches, la collaboration entre membres d'équipe ainsi que l'analyse des performances. La solution s'appuie sur la méthodologie agile **Scrum** et intègre l'assistant intelligent **Aura IA**, capable d'analyser les données des projets et de proposer des recommandations automatiques.

1.3 Problématique

La problématique exposée ici constitue le point de départ du projet et justifie la nécessité de concevoir une solution sur mesure plutôt que de recourir à des outils existants

du marché.

Dans un contexte professionnel de plus en plus compétitif, VAERDIA fait face à l'absence d'une solution unifiée pour piloter l'ensemble de ses activités projets. Les principales difficultés observées sont les suivantes :

- Perte de temps liée à la synchronisation manuelle entre outils non interopérables.
- Manque de visibilité sur l'avancement des projets, la charge des équipes et les coûts associés.
- Difficultés de communication et de collaboration dans les équipes réparties.
- Absence d'un reporting automatisé permettant une prise de décision rapide.
- Processus de suivi client peu structuré et gestion RH non intégrée.
- Absence totale d'automatisation et d'intelligence dans les processus.

Face à ces constats, la question centrale qui oriente ce projet est la suivante :

Comment concevoir et développer une plateforme interne unique, flexible et intelligente, capable de centraliser la gestion de projets Scrum, la collaboration d'équipe, le suivi client et le reporting, tout en intégrant une couche d'automatisation et d'intelligence artificielle pour assister proactivement les équipes et réduire la charge décisionnelle manuelle ?

1.4 Analyse de l'existant

Cette section dresse un état des lieux des pratiques actuelles au sein de VAERDIA et met en évidence les lacunes qui ont motivé la création d'une solution propriétaire.

1.4.1 Description de l'existant

À l'heure actuelle, VAERDIA coordonne ses projets à l'aide d'une variété d'outils non intégrés : tableurs Excel pour le suivi des tâches, applications de messagerie pour les échanges, services de stockage en ligne pour la documentation, et méthodes manuelles pour la gestion des congés et des demandes clients.

1.4.2 Critique de l'existant

L'analyse révèle plusieurs faiblesses structurelles : fragmentation et perte de temps estimée à 20–30% sur les activités administratives, manque de visibilité en temps réel, risques d'erreurs lors des synchronisations manuelles, absence totale d'automatisation et d'intelligence, et non-intégration de la gestion RH dans la planification projet.

1.4.3 Solution proposée

En réponse à ces observations, VAERDIA a choisi de créer en interne une plateforme intégrée pour la gestion de projets agiles, hébergée sur ses propres serveurs. Ce choix vise

à garantir la souveraineté des données, offrir une personnalisation complète et assurer une maîtrise des coûts sur le long terme. La plateforme se compose de huit modules fonctionnels : gestion des projets, gestion des tâches, gestion documentaire, communication interne, portail client et ticketing, gestion des congés, Aura IA, et reporting.

1.5 Solutions existantes

Afin de justifier le développement d’une solution propriétaire, nous présentons ici une analyse comparative des principaux outils de gestion de projet disponibles sur le marché, en évaluant leur adéquation avec les besoins de VAERDIA.



FIGURE 1.2 – Solutions de gestion de projet analysées sur le marché

Solution	Points forts	Limitations	Adéquation VAERDIA
Jira	Support avancé Scrum/Kanban, reporting détaillé	Interface complexe, coût élevé	Partielle
Trello	Interface intuitive, Kanban natif	Scrum non natif, reporting limité	Faible
Asana	Collaboration avancée	IA limitée, absence de module RH	Partielle
Monday.com	Dashboards visuels, automatisations	Moins adapté aux méthodologies agiles	Faible
ClickUp	Multi-vues, prix abordable	Configuration complexe, module RH absent	Partielle

TABLE 1.2 – Analyse comparative des solutions existantes sur le marché

Cette étude justifie pleinement le choix de développer une solution propriétaire, les outils du marché ne répondant pas aux besoins spécifiques de VAERDIA en termes de souveraineté des données, de module RH intégré et d’intelligence artificielle.

1.6 Méthodologie et formalisme adoptés

Cette section présente le cadre méthodologique retenu pour piloter le développement du projet, en expliquant les raisons du choix de Scrum et les rôles associés au sein de l'équipe projet.

1.6.1 Le choix de la méthodologie Scrum

Présentation de Scrum

Scrum est un cadre de travail agile conçu pour gérer et contrôler le développement de produits complexes. Il a été formalisé par Ken Schwaber et Jeff Sutherland au début des années 1990 [1]. Scrum repose sur trois piliers fondamentaux : la **transparence**, l'**inspection** et l'**adaptation**.

Le cycle Sprint

L'essence du Scrum repose sur le **Sprint**, un événement de durée limitée pendant lequel une version potentiellement livrable du produit est élaborée. Le cycle Sprint comprend quatre étapes : la Planification de Sprint, le Daily Scrum (15 minutes quotidiennes), la Revue de Sprint et la Rétrospective du Sprint.

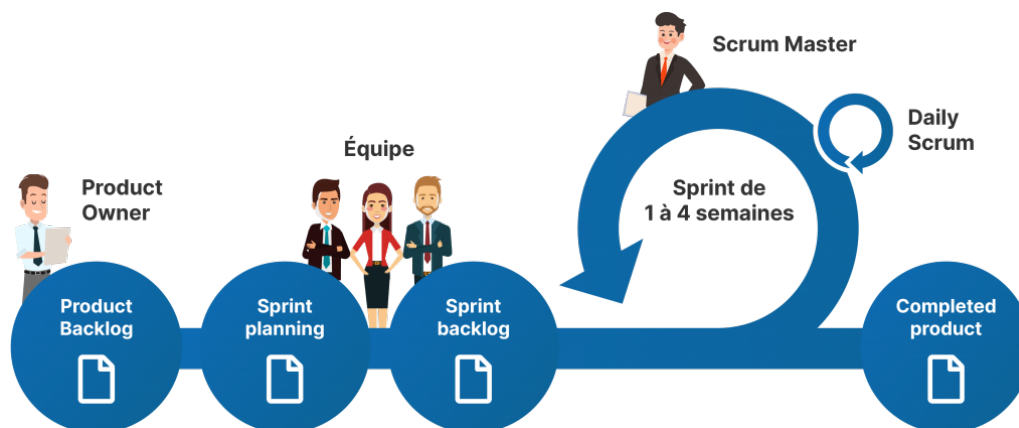


FIGURE 1.3 – Cycle de développement Scrum [1]

Les artefacts Scrum

Artefact	Description
Product Backlog	Liste ordonnée et dynamique de tout ce qui est nécessaire dans le produit, gérée par le Product Owner.
Sprint Backlog	Ensemble des éléments sélectionnés pour le Sprint, complété par un plan pour livrer l'Incrément.
Incrément	Somme de tous les éléments complétés durant un Sprint, devant respecter la Definition of Done pour être livrable.

TABLE 1.3 – Artefacts Scrum

Notre équipe Scrum

Rôle Scrum	Membre
Product Owner	Achref Makhoulfi
Scrum Master	Rania Mrad
Scrum Team	Eya Hermissi, Abdesslem Haouas

TABLE 1.4 – Composition de l'équipe Scrum du projet VAERDIA

Conclusion

Ce chapitre a permis d'établir une vue d'ensemble exhaustive du contexte dans lequel le projet VAERDIA s'inscrit. L'analyse critique des outils actuels et la détermination des besoins concrets des équipes ont révélé la nécessité de développer une solution centralisée, adaptable et intelligente. L'adoption de la méthode Scrum pour piloter le processus de développement a été justifiée. Le prochain chapitre se focalisera sur le pilotage du projet, en mettant en évidence les acteurs, les exigences, le backlog et la planification des sprints.

Chapitre 2

Pilotage du projet avec Agile Scrum

Introduction

Ce chapitre constitue le nœud fonctionnel du projet VAERDIA. Suite à l'établissement du contexte dans le chapitre précédent, nous entamons ici la phase de planification et d'organisation détaillée du projet. Nous commençons par l'identification des parties prenantes, puis nous élaborons le Product Backlog structuré en modules fonctionnels. Un diagramme de Gantt illustre la planification des releases et des sprints sur 16 semaines. Les exigences non fonctionnelles sont ensuite formalisées, avant de détailler l'architecture logicielle et physique retenue.

2.1 Identification des acteurs

Cette section identifie l'ensemble des parties prenantes du système VAERDIA et précise leurs droits d'accès respectifs, afin de poser les bases du modèle de contrôle d'accès basé sur les rôles (RBAC).

Acteur	Rôle principal	Droits et accès
Admin	Administration complète du système	CRUD complet, gestion utilisateurs et rôles RBAC, configuration globale, accès à tous les modules
Chef de projet	Pilotage et coordination des projets	Création et configuration de projets Scrum, gestion du backlog et des sprints, génération des rapports
Membre d'équipe	Exécution opérationnelle des tâches	Mise à jour des tâches, commentaires, import documents, chat projet, saisie du temps passé, congés
Client	Suivi externe du projet	Accès portail dédié, création et suivi de tickets, téléchargement des livrables
Aura IA	Assistance intelligente automatisée	Lecture seule sur les données projet, génération d'alertes, rapports hebdomadaires, réponses NLP

TABLE 2.1 – Identification et synthèse des acteurs du système VAERDIA

2.2 Product Backlog

Le Product Backlog représente la liste complète et hiérarchisée de toutes les fonctionnalités à réaliser. Il est organisé en modules fonctionnels, chaque module étant subdivisé en User Stories assorties de critères d'acceptation, d'un niveau de priorité et d'un sprint d'affectation.

2.2.1 Module : Authentification et accès

Ce module constitue le socle sécuritaire de la plateforme. Il garantit l'authenticité des utilisateurs et applique les permissions définies par le système RBAC.

ID	User Story	Critères d'acceptation	Priorité	Sprint
US-001	Authentification sécurisée avec JWT	Token JWT généré, session valide 24h, refresh token fonctionnel	Critique	S1
US-002	Gestion des rôles et permissions RBAC	5 rôles configurables, permissions par module vérifiées	Critique	S1
US-003	Création et gestion des comptes utilisateurs	CRUD utilisateurs complet, activation/désactivation opérationnels	Haute	S1

TABLE 2.2 – Backlog — Module Authentification et accès

2.2.2 Module : Gestion des projets

Ce module permet la création, la configuration et le suivi des projets Scrum, incluant la gestion des équipes, des budgets et des indicateurs de performance.

ID	User Story	Critères d'acceptation	Priorité	Sprint
US-004	Créer un projet avec données de cadrage	Projet créé avec équipe, budget, TJM et dates clés sauvegardés	Critique	S1
US-005	Affecter des membres à un projet	Affectation multiple, rôles par membre définis et modifiables	Critique	S1
US-006	Définir budget et TJM par ressource	Calcul automatique des coûts, modifiable à tout moment	Haute	S1
US-007	Gérer le statut des projets	Statuts : Planifié, En cours, Livré, Suspendu	Haute	S1
US-008	Consulter un dashboard KPIs simple	KPIs avancement, charge et coûts affichés en temps réel	Moyenne	S1

TABLE 2.3 – Backlog — Module Gestion des projets

2.2.3 Module : Gestion des tâches (Scrum)

Ce module implémente le cycle Scrum complet, depuis la gestion du backlog produit jusqu'à la clôture des sprints avec génération des rapports de vélocité.

ID	User Story	Critères d'acceptation	Priorité	Sprint
US-009	Gérer un backlog produit	Backlog créé, stories ajoutables, modifiables et supprimables	Critique	S2
US-010	Créer des User Stories avec critères d'acceptation	Stories créées, critères éditables au format structuré	Critique	S2
US-011	Estimer les stories en story points (Fibonacci)	Estimation sauvegardée et modifiable	Haute	S2
US-012	Prioriser les stories par drag & drop	Ordre modifiable par glisser-déposer dans le backlog	Haute	S2
US-013	Planifier un sprint	Sprint créé avec nom, dates, objectif et capacité définis	Critique	S2
US-014	Affecter des stories au sprint actif	Stories déplacées depuis le backlog, capacité vérifiée	Critique	S2
US-015	Déplacer des tâches sur le board Scrum	Drag & drop entre colonnes, statut mis à jour automatiquement	Critique	S2
US-016	Consulter le burndown chart	Chart mis à jour en temps réel, tendance et écart visibles	Haute	S2
US-017	Clôturer un sprint et générer un rapport de vélocité	Rapport stories terminées / restantes, export disponible	Haute	S2

TABLE 2.4 – Backlog — Module Gestion des tâches / Scrum

2.2.4 Module : Portail client, Documents & Communication

Ce module centralise les interactions avec les clients externes et facilite la collaboration interne grâce à la gestion documentaire et au chat en temps réel.

ID	User Story	Critères d'acceptation	Priorité	Sprint
US-018	Connexion au portail client sécurisé	Login sécurisé, accès restreint à mes projets	Critique	S3
US-019	Créer un ticket (Bug / Demande / Amélioration)	Type sélectionnable, description et pièces jointes supportées	Critique	S3
US-020	Suivre le statut de mes tickets	Statuts visibles, historique et notifications automatiques	Haute	S3
US-021	Importer et versionner des documents	Multi-format, versioning automatique v1/v2/v3	Haute	S3
US-022	Chat projet avec mentions @utilisateur	Messages temps réel WebSocket, auto-complétion @nom	Haute	S3
US-023	Consulter l'audit log du projet	Toutes les actions tracées et horodatées	Moyenne	S3

TABLE 2.5 – Backlog — Module Portail Client, Documents & Communication

2.2.5 Module : RH, Reporting & Aura IA

Ce module regroupe les fonctionnalités avancées de la plateforme, notamment la gestion des ressources humaines, la génération de rapports analytiques et l'intégration de l'assistant intelligent Aura IA.

ID	User Story	Critères d'acceptation	Priorité	Sprint
US-024	Soumettre et valider des demandes de congé	Formulaire avec dates, workflow de validation automatique	Moyenne	S4
US-025	Consulter un dashboard KPIs avancé	Tâches terminées, % avancement, retards, charge équipe	Critique	S4
US-026	Générer rapports d'avancement, charge et financiers	Export PDF et Excel avec graphiques inclus	Haute	S4
US-027	Recevoir des alertes IA si le sprint est en retard	Alerte automatique si burn-down < courbe idéale	Haute	S5
US-028	Interroger Aura IA en langage naturel	Réponse contextuelle en moins de 5 secondes	Moyenne	S5
US-029	Recevoir un rapport hebdomadaire automatique	Rapport synthétique généré et transmis chaque semaine	Haute	S5
US-030	Tests E2E et déploiement production	Couverture > 80%, déploiement Docker validé	Haute	S6

TABLE 2.6 – Backlog — Modules RH, Reporting, Aura IA & Déploiement

2.3 Planification des releases et sprints

Cette section décrit l'organisation temporelle du projet en releases et sprints, et précise les objectifs et livrables attendus à chaque étape du développement.

2.3.1 Principes de planification

Le projet est développé sur **16 semaines** (4 mois), organisé en **6 sprints** et en **deux releases** majeures. La Release 1 (R1) couvre les sprints S1 à S3 et livre le cœur du système ; la Release 2 (R2) couvre les sprints S4 à S6 et complète les fonctionnalités avancées jusqu'au déploiement en production. Chaque sprint est précédé d'une réunion de **Sprint Planning** et clôturé par une **Sprint Review** suivie d'une **Sprint Retrospective**.

2.3.2 Planification des releases

Rel.	Nom	Livrables clés
R1	Core System	Auth + RBAC, Gestion projets, Backlog, Sprints Scrum, Board drag & drop, Burndown Chart, Portail Client, Documents, Chat, Audit log
R2	Fonctionnalités Avancées	RH + Reporting + Export PDF/Excel, Aura IA (LangChain/NLP), Alertes intelligentes, Tests E2E, Déploiement production

TABLE 2.7 – Planification des deux releases du projet VAERDIA (16 semaines / 4 mois)

2.3.3 Planning détaillé des sprints

Sprint	Semaines	Fonctionnalités principales	Rel.
S1	Sem. 1–3	Authentification JWT, RBAC , CRUD utilisateurs, création projets, affectation membres, budget et TJM, dashboard KPIs simple	R1
S2	Sem. 4–6	Backlog produit (CRUD), User Stories avec critères, estimation Fibonacci, priorisation drag & drop, sprint planning, board Scrum, Burndown Chart (MVP), rapport de vélocité	
S3	Sem. 7–8	Portail client sécurisé, tickets (Bug / Demande / Amélioration), workflow de traitement, notifications email et in-app, versioning documents, chat + mentions @utilisateur, audit log	
S4	Sem. 9–11	Gestion des congés, calendrier projet, dashboard KPIs avancé, rapports d’avancement et financiers, statistiques RH, exports PDF / Excel	R2
S5	Sem. 12–14	Intégration LangChain + OpenAI, commandes NLP, alertes intelligentes, résumé automatique projets/sprints, rapport hebdomadaire automatique, analyse performances tâches & projets	
S6	Sem. 15–16	Tests E2E (>80 %), correction bugs, optimisation performances, documentation utilisateur, déploiement production	

TABLE 2.8 – Planning détaillé des 6 sprints — VAERDIA (16 semaines / 4 mois)

2.4 Diagramme de Gantt

Le diagramme suivant offre une vue macro de la planification du projet VAERDIA sur les 16 semaines de développement. Les activités sont regroupées par release afin de mettre en évidence la progression itérative et les jalons de livraison.

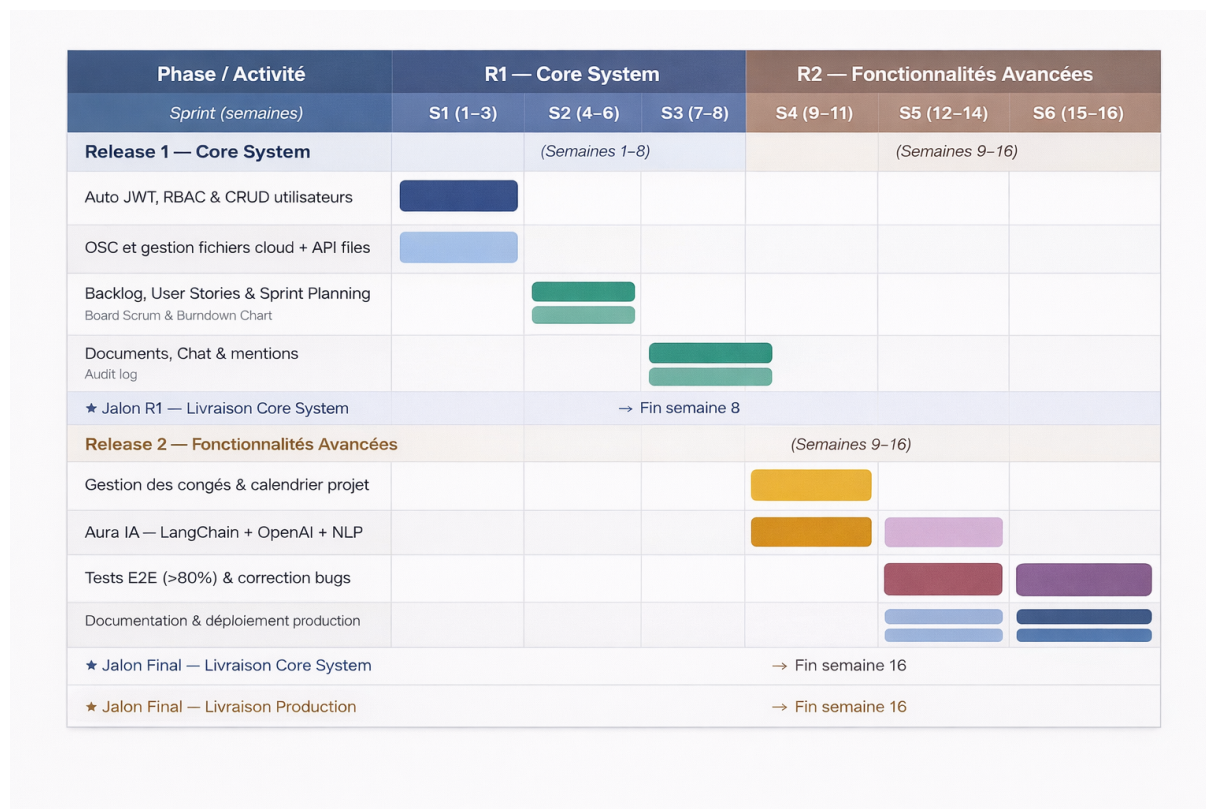


FIGURE 2.1 – Diagramme de Gantt — VAERDIA (16 semaines / 4 mois)

2.5 Besoins non fonctionnels

Les besoins non fonctionnels définissent les contraintes de qualité, de performance et de sécurité que le système doit respecter indépendamment des fonctionnalités métier. Ils constituent des critères de fiabilité et d'acceptation du système en conditions réelles.

Performance : Le système doit offrir des temps de réponse fluides, y compris pour les interactions en temps réel (chat, notifications).

Disponibilité : La plateforme doit rester accessible en continu durant les heures ouvrables, avec des interruptions limitées aux fenêtres de maintenance planifiées.

Sécurité : Authentification JWT avec refresh token, contrôle RBAC couvrant tous les rôles, communications chiffrées via HTTPS/TLS.

Maintenabilité : Architecture découplée permettant l'ajout de modules de façon indépendante, sans refactoring global.

Scalabilité et souveraineté : Déploiement via Docker Compose permettant une montée en charge horizontale ; données hébergées on-premise sur les serveurs internes.

Ergonomie : Interface responsive, compatible avec les navigateurs courants.

Qualité : Parcours fonctionnels critiques couverts par des tests End-to-End, documentation utilisateur complète fournie au déploiement.

2.6 Architecture logicielle et physique

Cette section décrit l'architecture technique retenue pour la plateforme VAERDIA, en présentant successivement la vue logicielle en couches et l'infrastructure physique de déploiement.

2.6.1 Architecture logicielle

VAERDIA adopte une architecture **microservices** organisée en cinq couches communiquant de manière unidirectionnelle, du client jusqu'à la persistance des données.

Frontend — Couche de présentation Développé en **React** (TypeScript), avec **shadcn/ui** pour la cohérence visuelle et **Zustand** pour la gestion de l'état global.

API Gateway (port 3000) Point d'entrée unique orchestrant le routage vers les microservices, appliquant les règles JWT + RBAC et gérant la communication temps réel via **WebSockets**.

Microservices métier (ports 3001–3006) et Aura IA (port 3007) Chaque domaine fonctionnel est isolé dans un service **Nest.js** indépendant. Le module **Aura IA** repose sur **FastAPI** (Python), exploitant **LangChain** et l'**API OpenAI**.

Persistance **PostgreSQL** assure la persistance relationnelle ; **MinIO** (S3-compatible) gère le stockage des fichiers.

Infrastructure DevOps Conteneurisation via **Docker Compose** ; pipeline **GitLab CI/CD** automatisant le build, les tests et le déploiement.

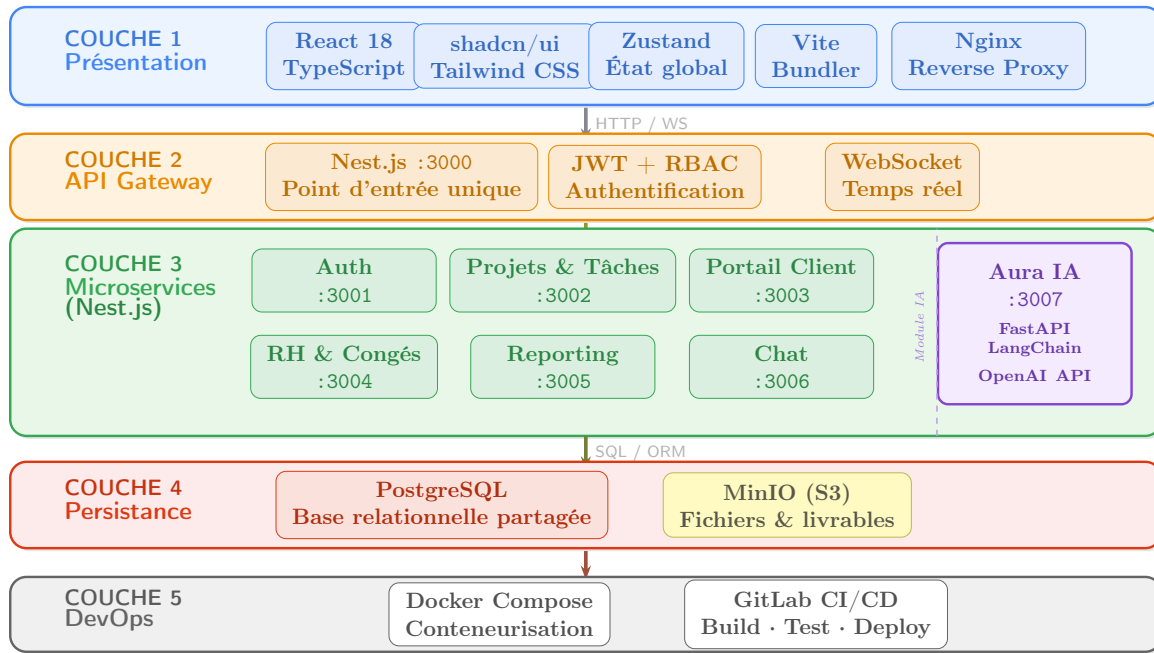


FIGURE 2.2 – Architecture logicielle de VAERDIA — vue en couches

2.6.2 Architecture physique

L'infrastructure physique est déployée en mode **on-premise** sur les serveurs internes de VAERDIA, garantissant la souveraineté totale des données. Cette approche évite toute dépendance à un fournisseur cloud externe et permet un contrôle complet des performances et de la sécurité.

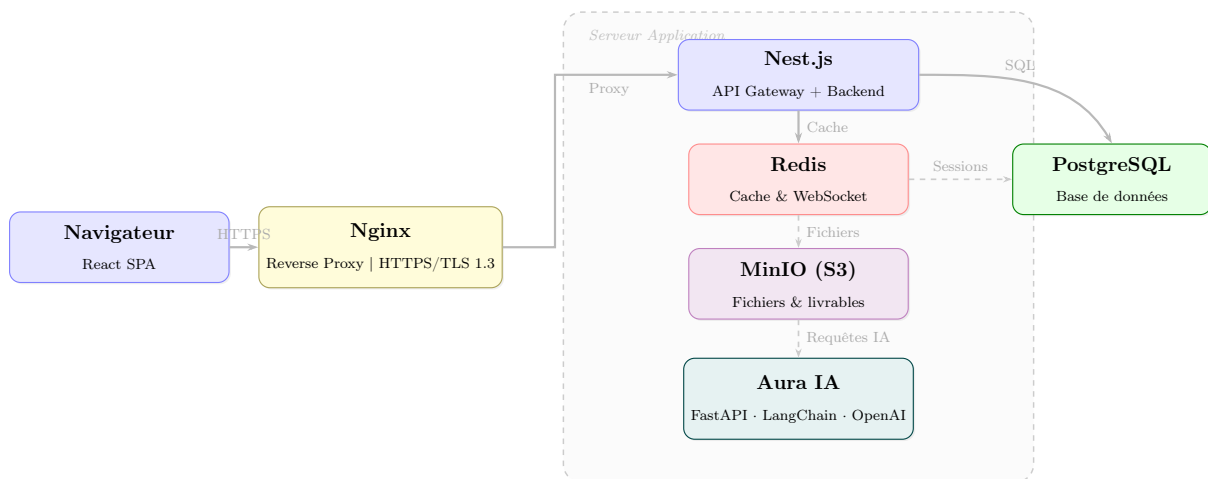


FIGURE 2.3 – Architecture physique de VAERDIA

2.7 Environnement technique de travail

Cette section recense l'ensemble des outils et technologies utilisés tout au long du projet, organisés par domaine (frontend, backend, collaboration), afin de constituer une référence complète de l'environnement de développement.

2.7.1 Environnement de développement Frontend

Le frontend de VAERDIA repose sur un ensemble de technologies modernes permettant de concevoir une interface utilisateur réactive, accessible et visuellement cohérente, tout en garantissant une gestion d'état performante côté client.

Logo	Outil / Technologie	Rôle
	React 18	Framework UI — composants et gestion du cycle de vie applicatif
	shadcn/ui	Bibliothèque de composants UI accessibles et personnalisables
	Tailwind CSS 3.x	Framework CSS utilitaire pour le design responsive
	Zustand 4.x	Gestion d'état global léger côté client
	VS Code	Éditeur de code principal pour le développement frontend

TABLE 2.9 – Environnement de développement Frontend

2.7.2 Environnement de développement Backend

Le backend s'appuie sur une architecture microservices orchestrée par Nest.js, complétée par un module IA en Python et une infrastructure de stockage et de persistance robuste, garantissant performance, scalabilité et souveraineté des données.

Logo	Outil / Technologie	Rôle
	Nest.js 10.x	API Gateway — orchestration des requêtes et routage
	FastAPI (Python)	Framework dédié exclusivement au module Aura IA
	Swagger / OpenAPI	Documentation automatique et test des endpoints REST
	PostgreSQL	Base de données relationnelle — persistance des données métier
	Redis	Cache applicatif, sessions et notifications temps réel
	MinIO (S3-compatible)	Stockage des fichiers, documents et pièces jointes
	LangChain + OpenAI API	Traitement du langage naturel pour Aura IA
	Docker + Docker Compose	Conteneurisation de l'ensemble des services

TABLE 2.10 – Environnement de développement Backend

2.7.3 Outils de collaboration et de gestion de projet

La coordination de l'équipe projet a été assurée grâce à une sélection d'outils dédiés à la gestion du code source, à la communication et à la modélisation, choisis pour leur complémentarité et leur intégration dans un workflow agile.



GitLab CI — Hébergement du code source, pipeline CI/CD automatisé, merge requests et revue de code.



GitHub — Sauvegarde et partage du dépôt distant en complément de GitLab.



Discord — Échanges quotidiens et coordination à distance.



Visual Paradigm — Conception des diagrammes UML.



Overleaf — Rédaction et compilation du rapport PFE en $\text{\LaTeX}$.



Lucidchart — est une plateforme de création de diagrammes et de visualisation de données en ligne, basée sur le cloud, qui permet de collaborer en temps réel [7].

Conclusion

Ce chapitre a posé les bases opérationnelles et techniques du projet VAERDIA. Le Product Backlog, structuré en 30 User Stories réparties sur 6 modules fonctionnels, constitue la feuille de route du projet. Sa planification en **6 sprints sur 16 semaines**, organisés en deux releases progressives (R1 : S1–S3, R2 : S4–S6), est illustrée par le diagramme de Gantt. Le chapitre suivant sera consacré à la conception détaillée et à la réalisation du Release 1.

Chapitre 3

Conception et réalisation du Release 1

Introduction

Ce chapitre présente la réalisation du Release 1, socle fondamental de la plateforme VAERDIA, structuré en trois sprints. La conception itérative et incrémentale est au cœur d’une démarche Scrum réussie. Le Sprint 1 couvre l’authentification JWT et le contrôle d’accès RBAC ; le Sprint 2 traite de la gestion des projets Scrum et du suivi budgétaire ; le Sprint 3 intègre le cycle Scrum complet, du backlog au burndown chart. Pour chaque sprint, nous présentons successivement le backlog partiel, les diagrammes UML de spécification, la conception et les interfaces réalisées.

3.1 Sprint 1 : Authentification

Ce sprint jette les fondations techniques de la plateforme en mettant en place le système d’authentification sécurisé et le contrôle d’accès basé sur les rôles, garantissant que chaque utilisateur n’accède qu’aux fonctionnalités qui lui sont autorisées.

3.1.1 Backlog du sprint 1

Les User Stories prioritaires traitées durant ce sprint sont présentées ci-dessous.

ID	User Story	Critères d’acceptation	Priorité
US-001	Authentification sécurisée avec JWT	Token JWT généré, session valide 24h, refresh token fonctionnel	Critique
US-002	Gestion des rôles RBAC	5 rôles configurables, permissions par module vérifiées	Critique
US-003	Gestion des comptes utilisateurs	CRUD utilisateurs complet, activation/désactivation opérationnels	Haute

TABLE 3.1 – Backlog du sprint 1 — Authentification & Infrastructure

3.1.2 Spécification des besoins du sprint 1

Cette sous-section formalise les besoins fonctionnels du sprint à travers des diagrammes de cas d'utilisation et des descriptions textuelles détaillant les scénarios nominaux et alternatifs.

Diagramme des cas d'utilisation

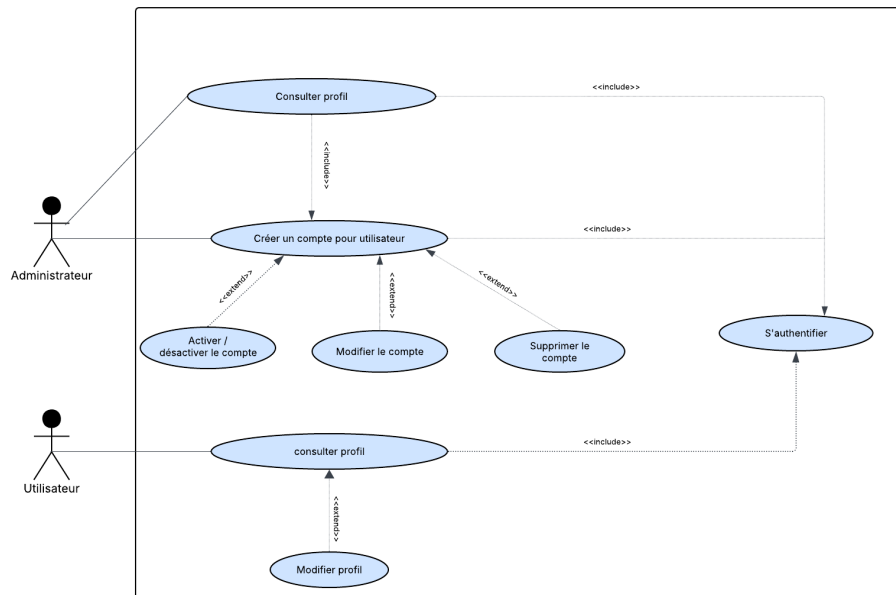


FIGURE 3.1 – Diagramme de cas d'utilisation — Sprint 1

Description textuelle des cas d'utilisation

Cas d'utilisation : Créer un utilisateur	
Acteur	Administrateur
Description	L'administrateur crée un nouveau compte utilisateur en renseignant ses informations et en lui attribuant un rôle.
Précondition	L'administrateur est authentifié. Aucun compte existant avec le même email.
Postcondition	Le compte est créé, le rôle RBAC est attribué et l'utilisateur reçoit ses identifiants.
Scénario principal	
Action de l'acteur	Réaction du système
1. L'admin accède à la gestion des utilisateurs.	2. Le système affiche la liste des utilisateurs existants.
3. L'admin remplit le formulaire (nom, email, rôle).	4. Le système vérifie la validité et l'unicité de l'email.
5. L'admin valide la création.	6. Le système crée le compte, attribue le rôle RBAC et génère le mot de passe initial.
Scénario alternatif	
Email déjà utilisé.	Le système affiche un message d'erreur de doublon.
Exception	
E1. Si des champs obligatoires sont manquants, le système bloque la soumission et indique les erreurs.	

TABLE 3.2 – Description textuelle — « Créer un utilisateur »

Cas d'utilisation « Créer un utilisateur »

Cas d'utilisation : Se connecter	
Acteur	Utilisateur
Description	L'utilisateur saisit ses identifiants pour accéder à son compte. Le système génère un token JWT stocké dans un cookie <code>HttpOnly</code> .
Précondition	Le compte utilisateur existe et a été créé par l'administrateur.
Postcondition	L'utilisateur est connecté et redirigé selon son rôle RBAC.
Scénario principal	
Action de l'acteur	Réaction du système
1. L'utilisateur accède à la page de connexion.	2. Le système affiche le formulaire de connexion.
3. L'utilisateur saisit ses identifiants.	4. Le système vérifie les credentials et génère un token JWT.
5. L'utilisateur valide.	6. Le système redirige vers le tableau de bord selon le rôle.
Scénario alternatif	
Mot de passe incorrect.	Le système affiche un message d'erreur.
Exception	
E1. Serveur indisponible : message d'erreur affiché. E2. Cookie JWT expiré : redirection vers la connexion.	

TABLE 3.3 – Description textuelle — « Se connecter »

Cas d'utilisation « Se connecter »

3.1.3 Conception du sprint 1

Cette sous-section présente les diagrammes de conception élaborés pour modéliser la structure et le comportement du système lors du sprint 1, en s'appuyant sur les notations UML standards.

Diagramme de classes du sprint 1

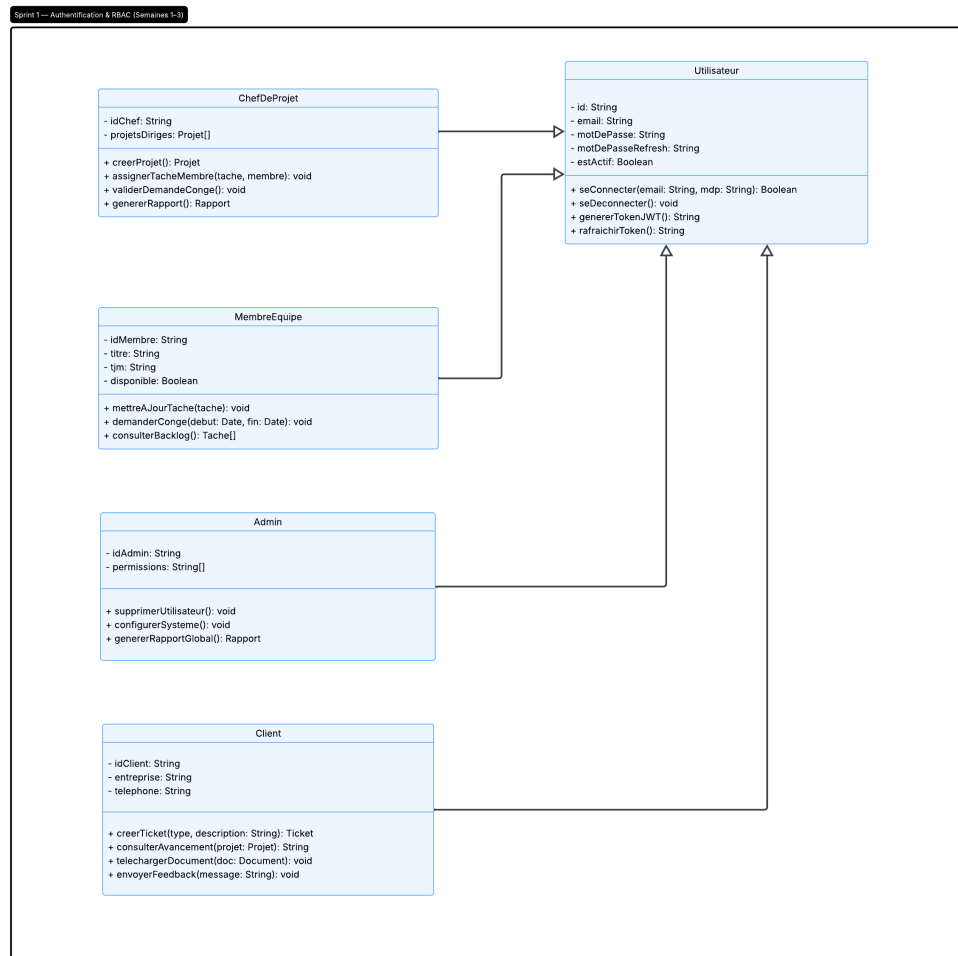


FIGURE 3.2 – Diagramme de classes du sprint 1

Diagramme de séquence : Créer un utilisateur

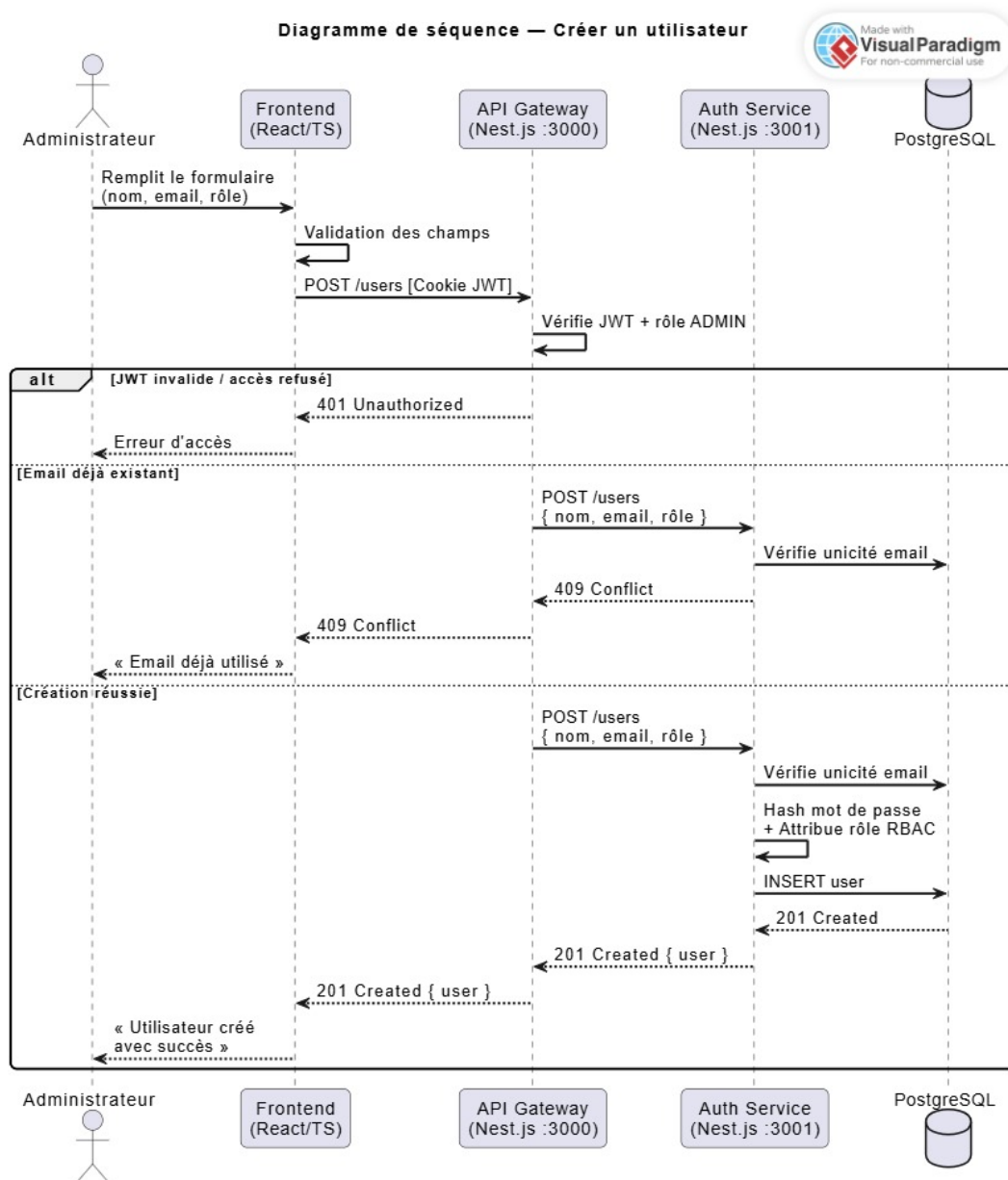


FIGURE 3.3 – Diagramme de séquence « Créer un utilisateur »

Diagramme de séquence : Se connecter

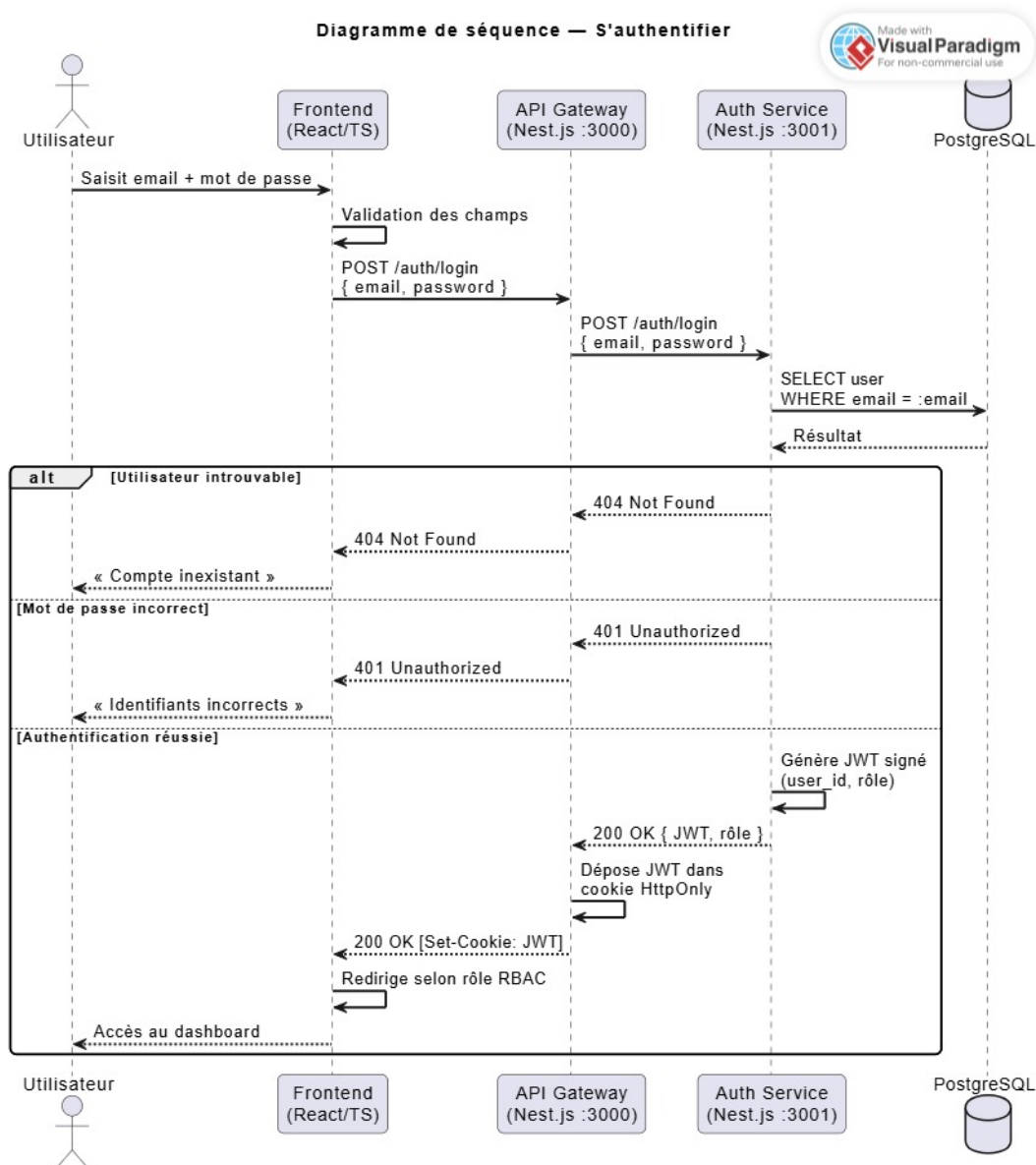


FIGURE 3.4 – Diagramme de séquence « Se connecter »

3.1.4 Implémentation du sprint 1

Cette sous-section illustre les interfaces utilisateur réalisées à l'issue du sprint 1, correspondant aux fonctionnalités d'authentification et de gestion des comptes telles que définies dans le backlog.

— Interface d'accueil :



FIGURE 3.5 – Interface « Accueil »

— **Interface de connexion :**



Images/sprint1/ui_login.png

FIGURE 3.6 – Interface « Se connecter »

— **Interface « Gestion des utilisateurs » (Admin) :**



FIGURE 3.7 – Interface « Gestion des utilisateurs »

3.2 Sprint 2 : Gestion des projets

Ce sprint introduit les fonctionnalités centrales de création et de suivi des projets Scrum, permettant aux chefs de projet de configurer l'environnement de travail, de constituer les équipes et d'assurer un suivi budgétaire en temps réel.

3.2.1 Backlog du sprint 2

ID	User Story	Critères d'acceptation	Priorité
US-004	Créer un projet avec données de cadrage	Projet créé avec équipe, budget, TJM et dates sauvegardés	Critique
US-005	Affecter des membres à un projet	Affectation multiple, rôles par membre modifiables	Critique
US-006	Définir budget et TJM par ressource	Calcul automatique des coûts, modifiable à tout moment	Haute
US-007	Gérer les statuts des projets	Statuts : Planifié, En cours, Livré, Suspendu	Haute
US-008	Consulter un dashboard KPIs simple	KPIs avancement, charge et coûts en temps réel	Moyenne

TABLE 3.4 – Backlog du sprint 2 — Gestion des projets

3.2.2 Spécification des besoins du sprint 2

Les besoins de ce sprint sont formalisés à travers un diagramme de cas d'utilisation et une description textuelle détaillant le flux de création d'un projet Scrum par un chef de projet authentifié.

Diagramme des cas d'utilisation

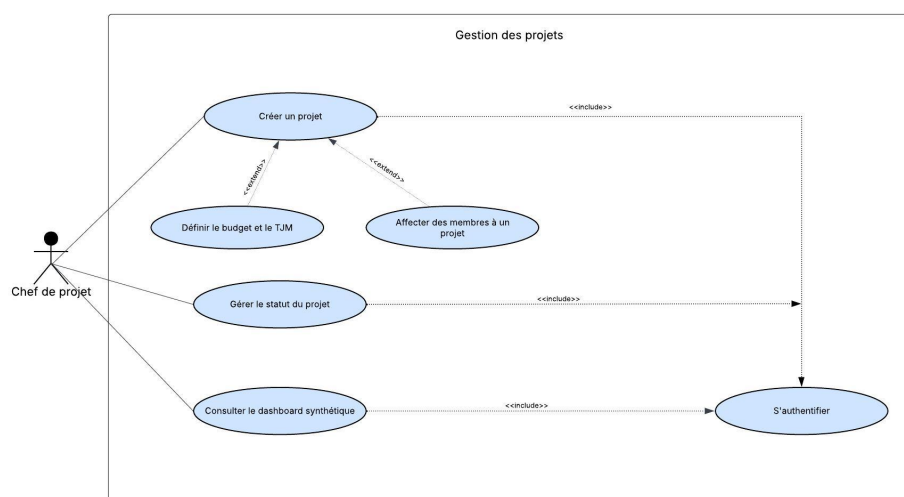


FIGURE 3.8 – Diagramme de cas d'utilisation « Gérer les projets »

Description textuelle : Créer un projet

Cas d'utilisation : Créer un projet	
Acteur	Chef de projet
Description	Le chef de projet crée un nouveau projet Scrum en renseignant ses informations de cadrage (nom, équipe, budget, TJM, dates).
Précondition	L'utilisateur est authentifié avec le rôle Chef de projet.
Postcondition	Le projet est créé et visible dans le tableau de bord.
Scénario principal	
Action de l'acteur	Réaction du système
1. Accès au formulaire de création.	2. Affichage du formulaire de cadrage.
3. Saisie des informations (nom, budget, membres).	4. Vérification de la cohérence des données.
5. Validation de la création.	6. Création du projet et calcul automatique des coûts.
Scénario alternatif	
Budget négatif ou incohérent.	Message d'erreur de validation.
Exception	
E1. Champs obligatoires manquants : soumission bloquée avec indication des champs à corriger.	

TABLE 3.5 – Description textuelle — « Créer un projet »

3.2.3 Conception du sprint 2

La modélisation de ce sprint repose sur un diagramme de classes décrivant les entités métier (Projet, Membre, Budget) et leurs relations, ainsi qu'un diagramme de séquence illustrant le flux complet de création d'un projet.

Diagramme de classes du sprint 2

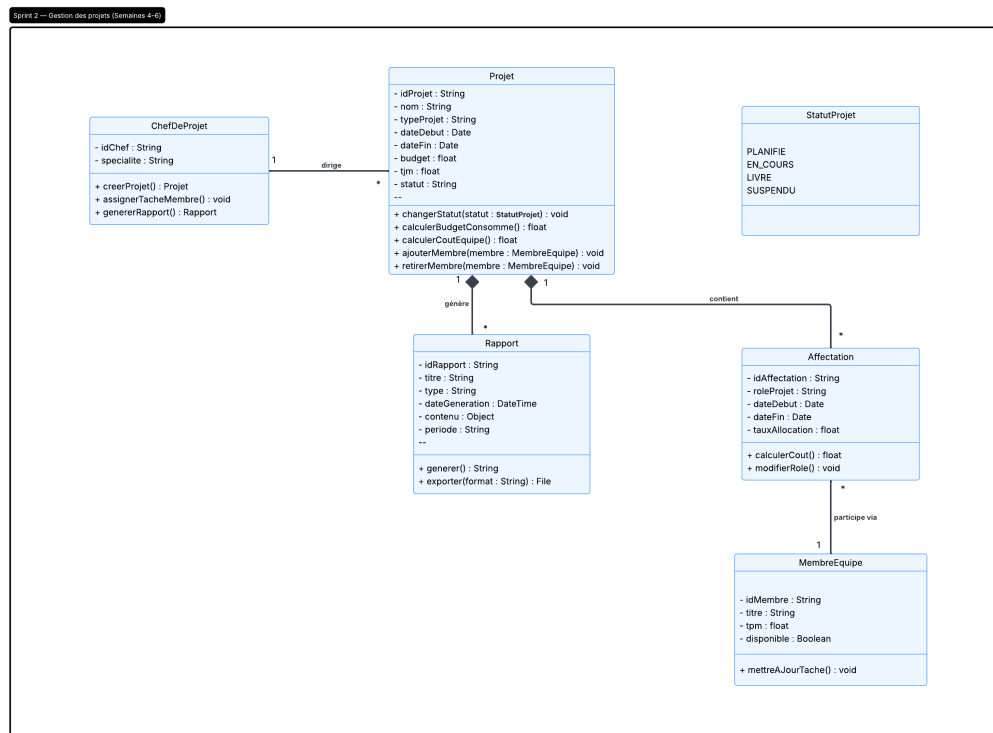


FIGURE 3.9 – Diagramme de classes du sprint 2

Diagramme de séquence : Créer un projet

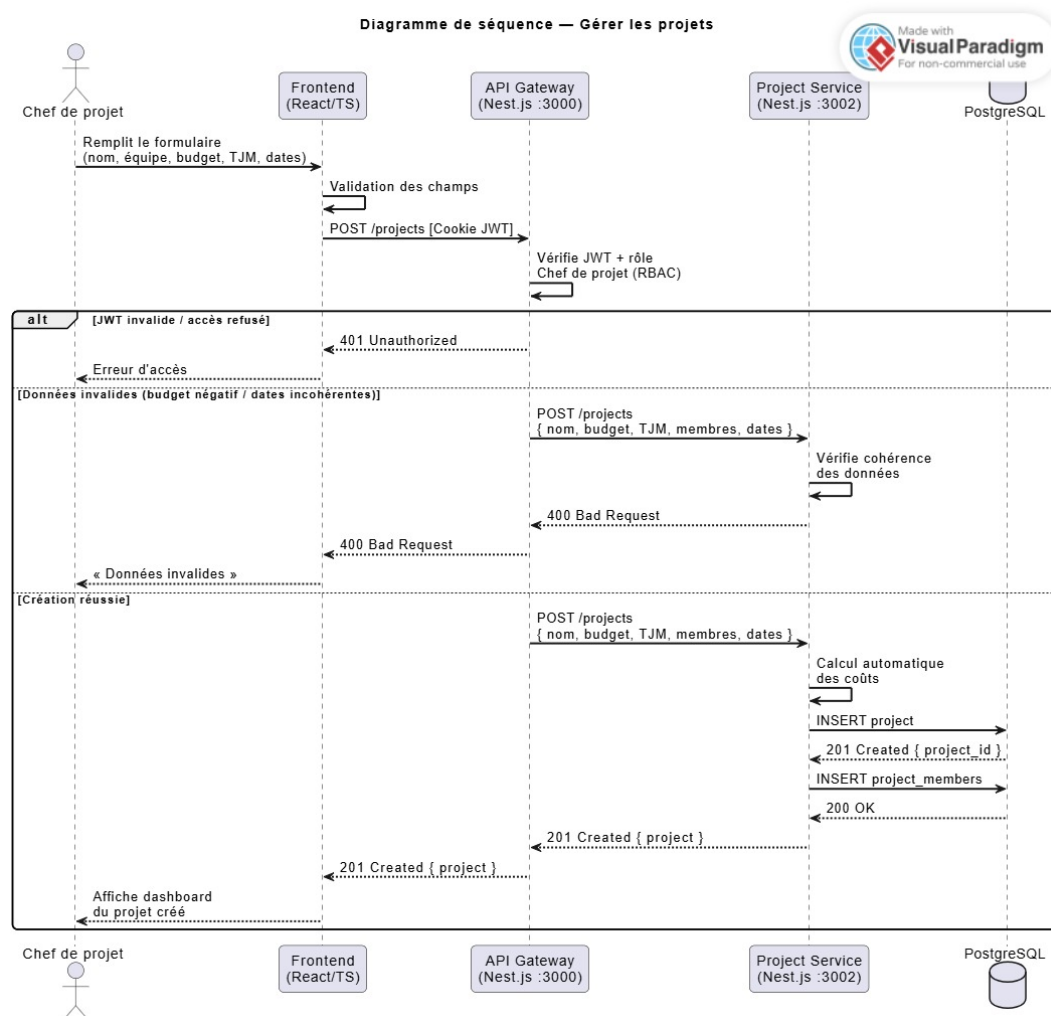


FIGURE 3.10 – Diagramme de séquence « Créer un projet »

3.2.4 Implémentation du sprint 2

Les captures suivantes illustrent les interfaces développées au cours du sprint 2, couvrant la création de projets et la consultation du tableau de bord des indicateurs de performance.

— Interface « Créer un projet » :



FIGURE 3.11 – Interface « Créer un projet »

— **Interface « Dashboard projets » :**



FIGURE 3.12 – Interface « Dashboard projets »

3.3 Sprint 3 : Backlog & Scrum

Ce sprint complète le Release 1 en implémentant le cycle Scrum dans son intégralité : création et priorisation du backlog, planification des sprints, board interactif avec drag & drop, et génération automatique du burndown chart.

3.3.1 Backlog du sprint 3

ID	User Story	Critères d'acceptation	Priorité
US-009	Gérer un backlog produit	Backlog créé, stories ajoutables, modifiables et supprimables	Critique
US-010	Créer des User Stories avec critères d'acceptation	Stories créées, critères éditables au format structuré	Critique
US-011	Estimer les stories (Fibonacci)	Estimation sauvegardée et modifiable	Haute
US-012	Prioriser par drag & drop	Ordre modifiable dans le backlog	Haute
US-013	Planifier un sprint	Sprint créé avec nom, dates, objectif et capacité	Critique
US-014	Affecter des stories au sprint actif	Stories déplacées, capacité vérifiée	Critique
US-015	Board Scrum drag & drop	Statut mis à jour automatiquement	Critique
US-016	Burndown chart du sprint actif	Chart temps réel, tendance et écart visibles	Haute
US-017	Clôturer un sprint	Rapport de vélocité généré et exportable	Haute

TABLE 3.6 – Backlog du sprint 3 — Backlog & Scrum

3.3.2 Spécification des besoins du sprint 3

Les besoins de ce sprint sont organisés en deux blocs distincts : la gestion du backlog produit d'une part, et la planification et l'exécution des sprints d'autre part, chacun modélisé par un diagramme de cas d'utilisation dédié.

Diagramme des cas d'utilisation — Gérer le backlog

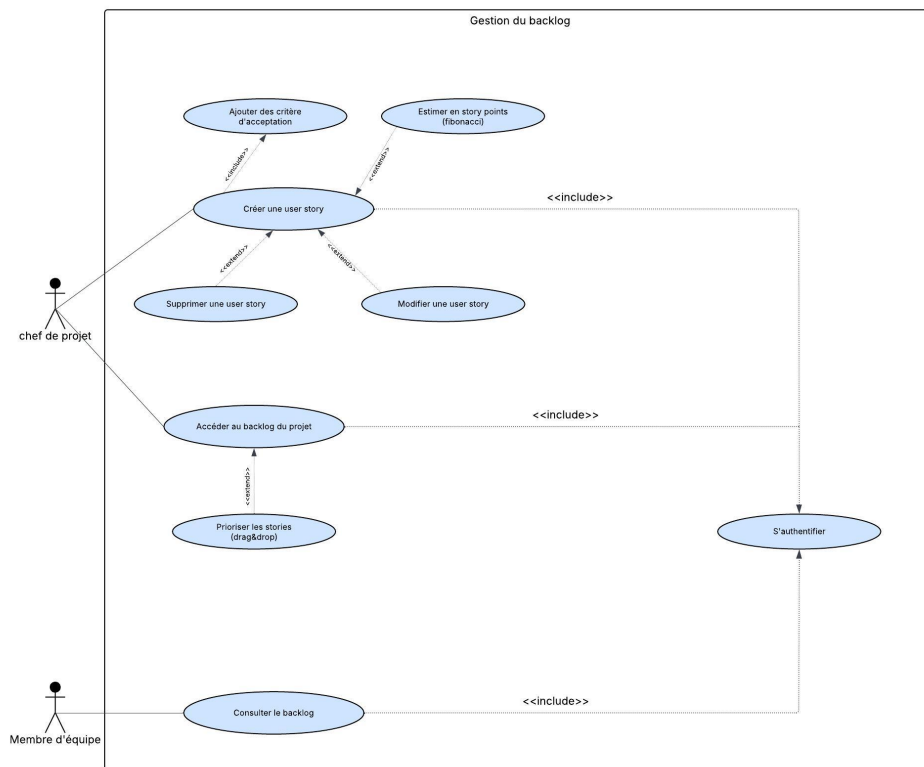


FIGURE 3.13 – Diagramme de cas d'utilisation « Gérer le backlog »

Diagramme des cas d'utilisation — Gérer un sprint

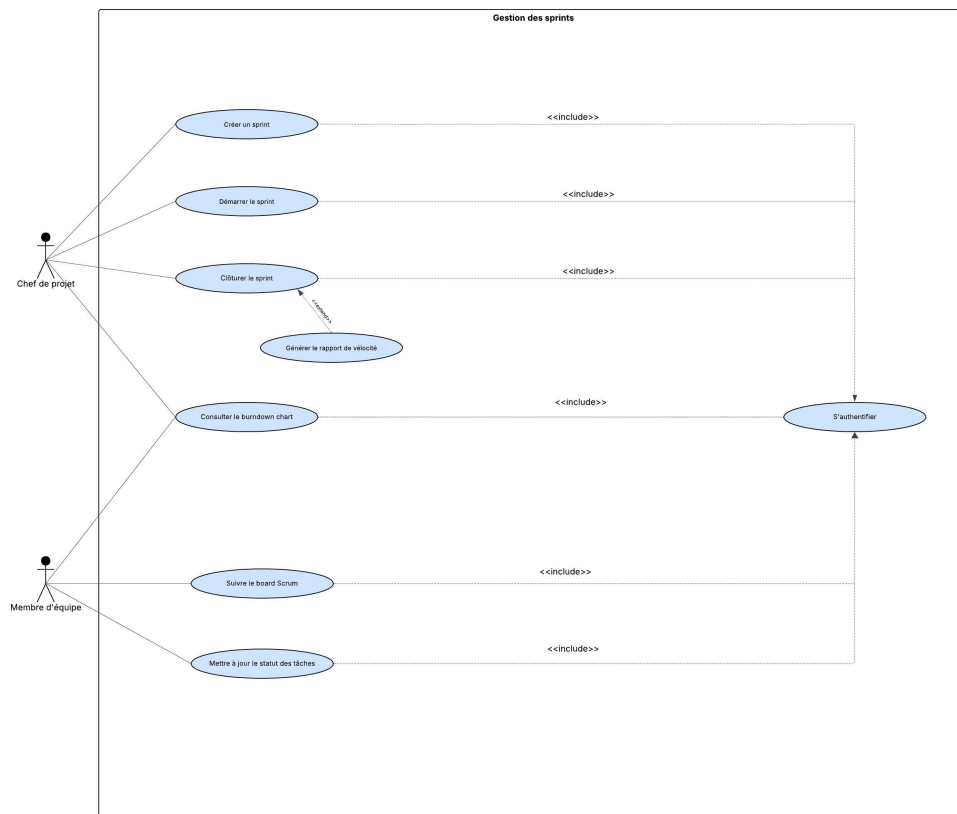


FIGURE 3.14 – Diagramme de cas d'utilisation « Gérer un sprint »

Description textuelle : Gérer le backlog

Cas d'utilisation : Gérer le backlog	
Acteur	Chef de projet
Description	Le chef de projet crée et gère les User Stories du Product Backlog avec leurs critères et estimations.
Précondition	Un projet Scrum existe.
Postcondition	Le backlog est mis à jour et les stories sont priorisées.
Scénario principal	
Action de l'acteur	Réaction du système
1. Accès au backlog du projet.	2. Affichage des User Stories.
3. Création d'une User Story avec critères.	4. Enregistrement et ajout au bas du backlog.
5. Estimation en story points.	6. Sauvegarde de l'estimation.
7. Réordonnancement par drag & drop.	8. Mise à jour de la priorité.
Scénario alternatif	
Suppression d'une story affectée à un sprint.	Le système demande confirmation avant suppression.
Exception	
E1. Description vide : création bloquée.	

TABLE 3.7 – Description textuelle — « Gérer le backlog »

Description textuelle : Planifier un sprint

Cas d'utilisation : Planifier un sprint	
Acteur	Chef de projet
Description	Le chef de projet crée un sprint, y affecte des User Stories et le démarre pour l'équipe.
Précondition	Un backlog avec stories estimées existe.
Postcondition	Le sprint est actif et le board Scrum est accessible.
Scénario principal	
Action de l'acteur	Réaction du système
1. Création du sprint avec nom, dates et objectif.	2. Initialisation du sprint et calcul de la capacité.
3. Sélection des stories depuis le backlog.	4. Vérification de la capacité.
5. Démarrage du sprint.	6. Activation du board et initialisation du burndown.
Scénario alternatif	
Capacité dépassée.	Avertissement et suggestion de retrait de stories.
Exception	
E1. Dates incohérentes : création bloquée.	

TABLE 3.8 – Description textuelle — « Planifier un sprint »

3.3.3 Conception du sprint 3

Le diagramme de classes du sprint 3 modélise les entités Scrum (Backlog, UserStory, Sprint, Board, BurndownChart) et leurs associations, reflétant la structure de données implémentée dans les microservices correspondants.

Diagramme de classes du sprint 3

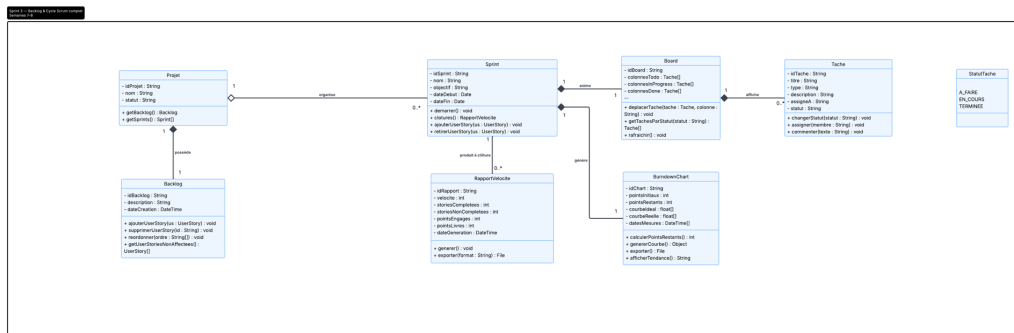
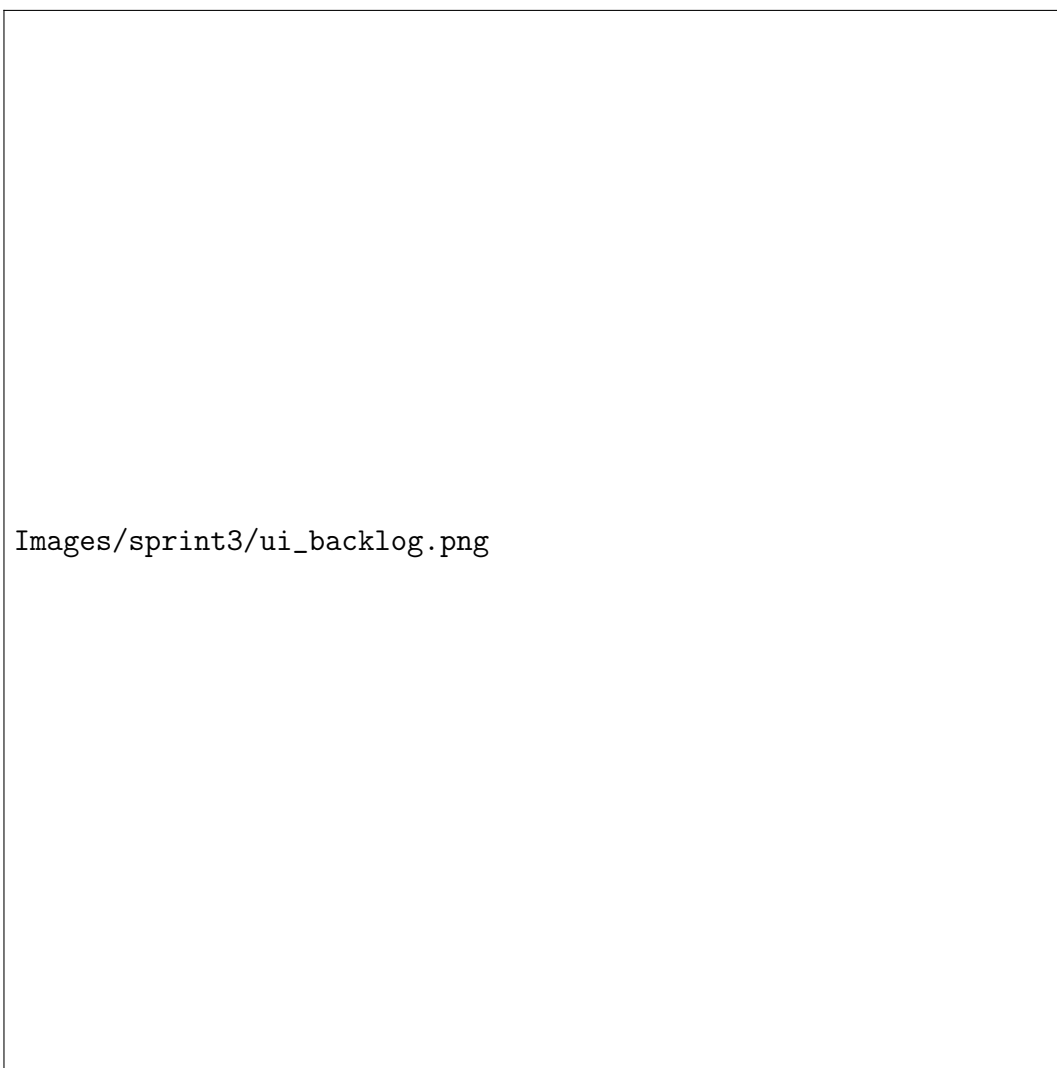


FIGURE 3.15 – Diagramme de classes du sprint 3

3.3.4 Implémentation du sprint 3

Les captures d'écran suivantes illustrent les fonctionnalités Scrum livrées à l'issue du sprint 3, notamment le board interactif, le burndown chart en temps réel et le mécanisme de clôture de sprint avec génération du rapport de vélocité.

— Interface « Product Backlog » :



Images/sprint3/ui_backlog.png

FIGURE 3.16 – Interface « Product Backlog »

— Interface « Créer une User Story » :



FIGURE 3.17 – Interface « Créer une User Story »

— Interface « Board Scrum » :

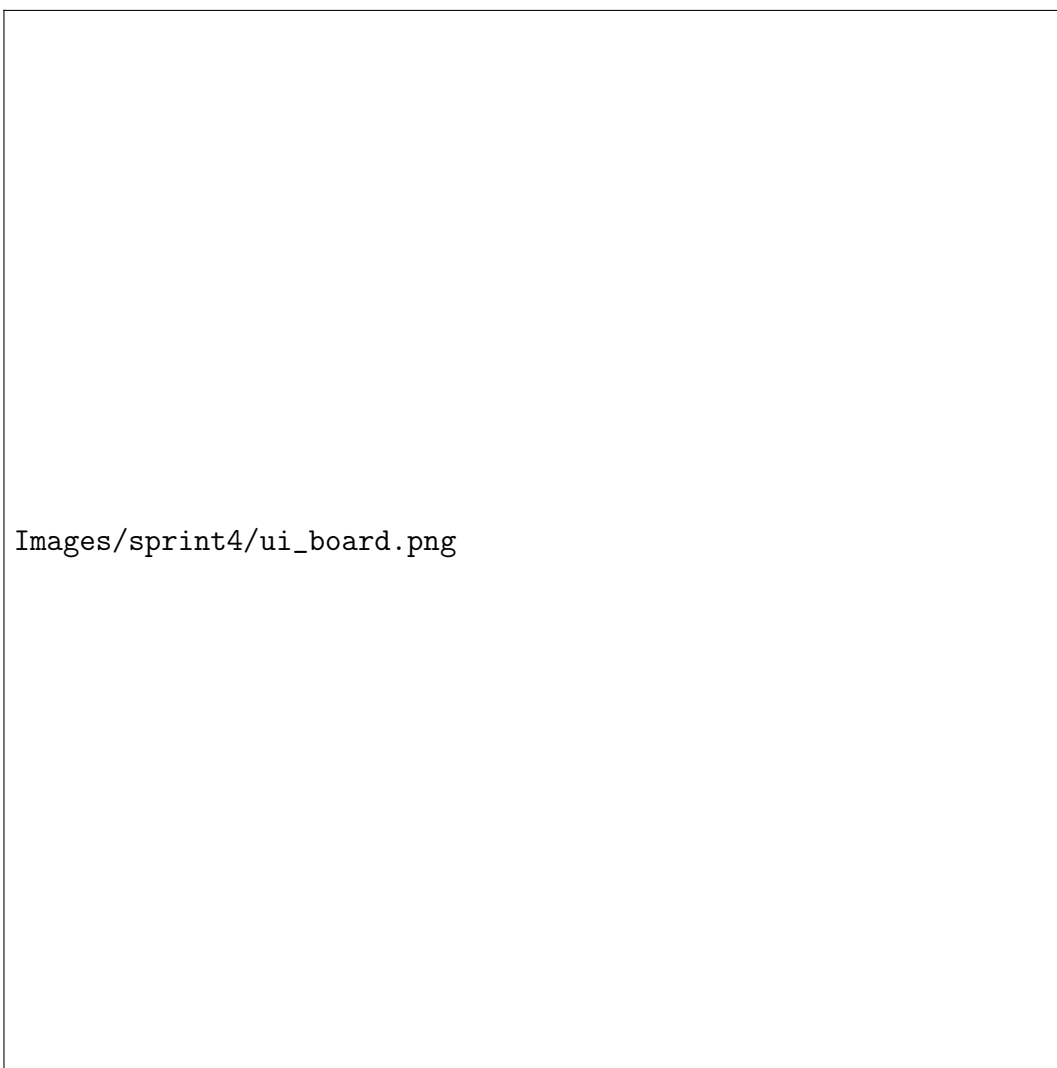


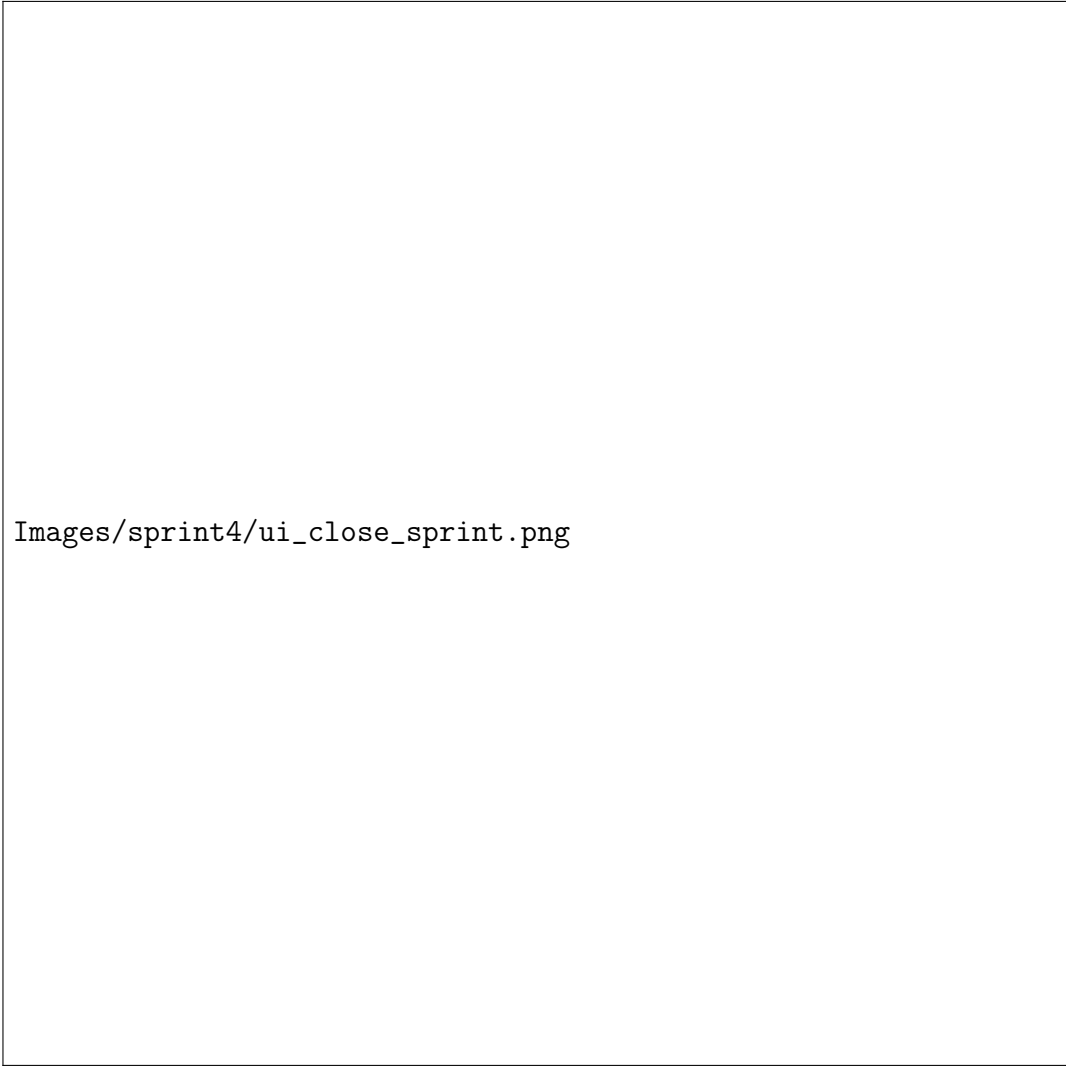
FIGURE 3.18 – Interface « Board Scrum »

— **Interface « Burndown Chart » :**



FIGURE 3.19 – Interface « Burndown Chart »

— **Interface « Clôture de sprint » :**

The image is a large rectangular box representing a screenshot of a software interface. Inside the box, the text 'Images/sprint4/ui_close_sprint.png' is displayed in a monospaced font, indicating the location of the image file. The rest of the box is empty, suggesting the actual interface content is not visible or is obscured by the text.

Images/sprint4/ui_close_sprint.png

FIGURE 3.20 – Interface « Clôture de sprint »

Conclusion

Ce chapitre a présenté la conception et la réalisation du premier release du projet VAERDIA, qui constitue le socle fondamental de la plateforme. Les trois sprints développés couvrent : l'authentification sécurisée JWT avec RBAC (Sprint 1), la création et le pilotage des projets Scrum avec tableau de bord et suivi budgétaire (Sprint 2), ainsi que le cycle Scrum complet comprenant backlog, board interactif et burndown chart automatique (Sprint 3). Le chapitre suivant portera sur le Release 2, détaillant les fonctionnalités avancées : portail client, gestion documentaire, communication interne, RH, reporting et Aura IA.

Bibliographie

- [1] Schwaber, K. et Sutherland, J. *Le Guide Scrum — La règle du jeu*. Boston : Scrum.org, 2020, 19 p. Disponible sur : <https://scrumguides.org/scrum-guide.html> [Consulté le 10 février 2024].
- [2] Addinn Group. *Scrum : une méthodologie agile indispensable pour le développement logiciel moderne*. En ligne : Addinn Group, 20 août 2023. Disponible sur : <https://addinn-group.com/2023/08/20/scrum-une-methodologie-agile-indispensable-pour-le-developpement-logiciel-moderne/> [Consulté le 15 mars 2024].
- [3] Meta Platforms, Inc. *React — Documentation officielle*. En ligne : Meta Platforms, 2024. Disponible sur : <https://react.dev/> [Consulté le 5 avril 2024].
- [4] Myśliwiec, K. *NestJS — A progressive Node.js framework*. En ligne : NestJS, 2024. Disponible sur : <https://docs.nestjs.com/> [Consulté le 5 avril 2024].
- [5] LangChain, Inc. *LangChain Documentation*. En ligne : LangChain, 2024. Disponible sur : <https://python.langchain.com/docs/> [Consulté le 20 mars 2024].
- [6] Mannai, R. *Conception et développement d'une plateforme de gestion de projets agiles*. Rapport de projet de fin d'études. École Polytechnique de Sousse : 2024.
- [7] Lucid Software, Inc. *Lucidchart — Plateforme de diagrammes et de visualisation en ligne*. En ligne : Lucid Software, 2024. Disponible sur : <https://www.lucidchart.com> [Consulté le 18 avril 2026].